

Digital IC Design
Physical Design Lab
Macro Integration in Backend Design

Name	*Sidra Naveed*
Instructor	*Hira Sohail*
Date	*14th October, 2025*

Introduction

In modern VLSI design, complex systems often require large on-chip memories such as SRAMs or ROMs.Instead of designing these memories from scratch, semiconductor foundries provide pre-designed memory macros (IP blocks) that are already optimized for area, speed, and power.This lab focuses on the integration of a Dual-Port SRAM (DP-RAM) macro into a digital design flow.The Dual-Port RAM allows simultaneous read/write operations on two independent ports, making it suitable for high-performance data-sharing applications such as processors, buffers, and communication systems.

Students perform both front-end (RTL design and synthesis) and back-end (floorplanning, placement, and routing) steps to understand how a pre-designed macro is added, timed, and verified within a full chip design.

Front End:

Script File:

```
# ================================================
# Project Setup
# ================================================
set DESIGN           dual_port_ram                ;# Top module name
set GEN_EFF          high                         ;# Generic synthesis effort
set MAP_OPT_EFF      high                         ;# Mapping + optimization effort
set LIB_PATH        /home/cc/Downloads/Lab_05_Materials/macro_lab/mmmc_files/libs
set RTL_PATH        /home/cc/Downloads/Lab_05_Materials/macro_lab/RTL
set SDC_FILE        /home/cc/Downloads/Lab_05_Materials/macro_lab
set NETLIST_DIR     ./Netlist
set OUTPUT_DIR      ./out
```

Defines paths and important project variables:

- DESIGN → top-level module name.
- LIB_PATH → where the standard cell and macro libraries are stored.
- RTL_PATH → folder containing your Verilog/SystemVerilog source files.
- SDC_FILE → folder with your constraint file.
- NETLIST_DIR & OUTPUT_DIR → where Genus will save output files.

```
# ================================================
# RTL Read & Elaboration
# ================================================
read_hdl -language sv $RTL_PATH/dual_port_ctrl.sv
read_hdl -language sv $RTL_PATH/macro_stub.sv

# Tell Genus not to do any optimization on macro
set_dont_touch TSDN65LPLLA1024X32M4S
```

- Reads your **SystemVerilog** source files. dual_port_ctrl.sv is your main controller logic. macro_stub.sv is the **black-box model** for your hard macro (Dual-Port RAM).

```
# ================================================
# Constraints
# ================================================
read_sdc $SDC_FILE/constraint.sdc

# ================================================
# Synthesis: Generic
# ================================================
set_db / .syn_generic_effort $GEN_EFF
syn_generic
ungroup -all -flatten

# ================================================
# Synthesis: Mapping
# ================================================
set_db / .syn_map_effort $MAP_OPT_EFF
syn_map

# ================================================
# Synthesis: Optimization
# ================================================
set_db / .syn_opt_effort $MAP_OPT_EFF
syn_opt

# ================================================
# Netlist & Constraints Output
# ================================================
write_hdl $DESIGN -mapped > $NETLIST_DIR/${DESIGN}_map.v
write_sdc $DESIGN > $OUTPUT_DIR/${DESIGN}_map.sdc
write_sdf -design $DESIGN > $OUTPUT_DIR/${DESIGN}_map.sdf
```

Reads your **timing constraint file (.sdc)**. Contains clock definition, input/output delays, and other timing info.

Generic synthesis: Translates Verilog into a technology-independent representation.
Mapping: Replaces generic gates with actual cells from the .lib file.
Optimization: Refines the design for better performance and area.

Explanation of the Synthesis Script

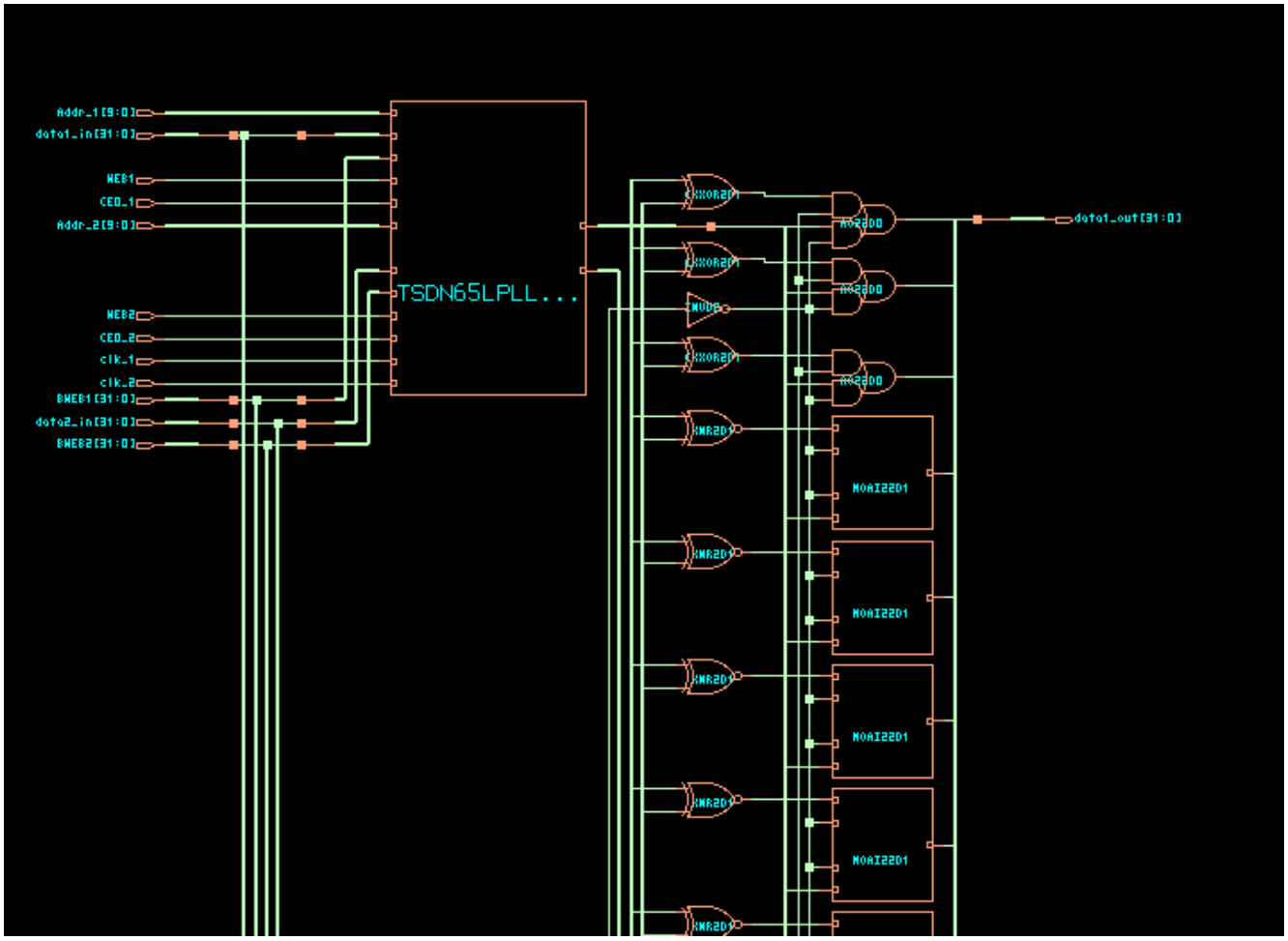
In this step, I used a synthesis script in Cadence Genus to convert my RTL design into a gate-level netlist. The script first checks the system information and then sets up paths for libraries, RTL files, and output folders. It reads the standard cell libraries and my Verilog source files, including the macro stub, which is treated as a black-box so it is not modified during synthesis. The timing constraints file is then applied

to define the clock and input/output delays. After that, the tool performs generic synthesis, mapping, and optimization to generate a technology-mapped version of the design. Finally, the script writes out the synthesized netlist, updated constraints, and timing data files. It also generates various reports such as timing, area, power, and quality of results. These outputs are later used for the physical design stage in Cadence Innovus.

Constraint File:

In this file, I defined two asynchronous clocks (`clk_1` and `clk_2`) with a period of **5.87 ns** to set the timing environment for my design. I added small clock uncertainties and transition times to model realistic behavior. Input and output delays of **1.75 ns** were applied to represent external timing effects. These constraints guide the synthesis tool to meet proper setup and hold timing requirements for reliable operation.

Netlist:



Post Synthesis:

xrun -timescale 1ns/10ps dual_port_ram_map.v macro_stub.sv dp_ram_test.sv dual_port_ctrl.sv -v tpzn65lpgv2od3.v tcbn65lp.v -access +rwc -mess -gui

```
  Building instance specific data structures.
  Loading native compiled code:     ..................... Done
  Design hierarchy summary:
                             Instances  Unique
          Modules:                 973     848
          UDPs:                   7552       4
          Primitives:            10474      10
          Timing outputs:         1136     759
          Registers:               374     376
          Scalar wires:           4493       -
          Vectored wires:           14       -
          Always blocks:            59      59
          Initial blocks:            2       2
          Cont. assignments:         2       2
          Pseudo assignments:        8       -
          Timing checks:         19120       -
          Process Clocks:           56      56
          Simulation timescale:    1ps
  Writing initial simulation snapshot: worklib.dual_port_ram:v
```

```
Exceptions/Constraints:
  input_delay             1750           constraint.sdc_line_44
  output_delay            1750           constraint.sdc_line_54_310_1

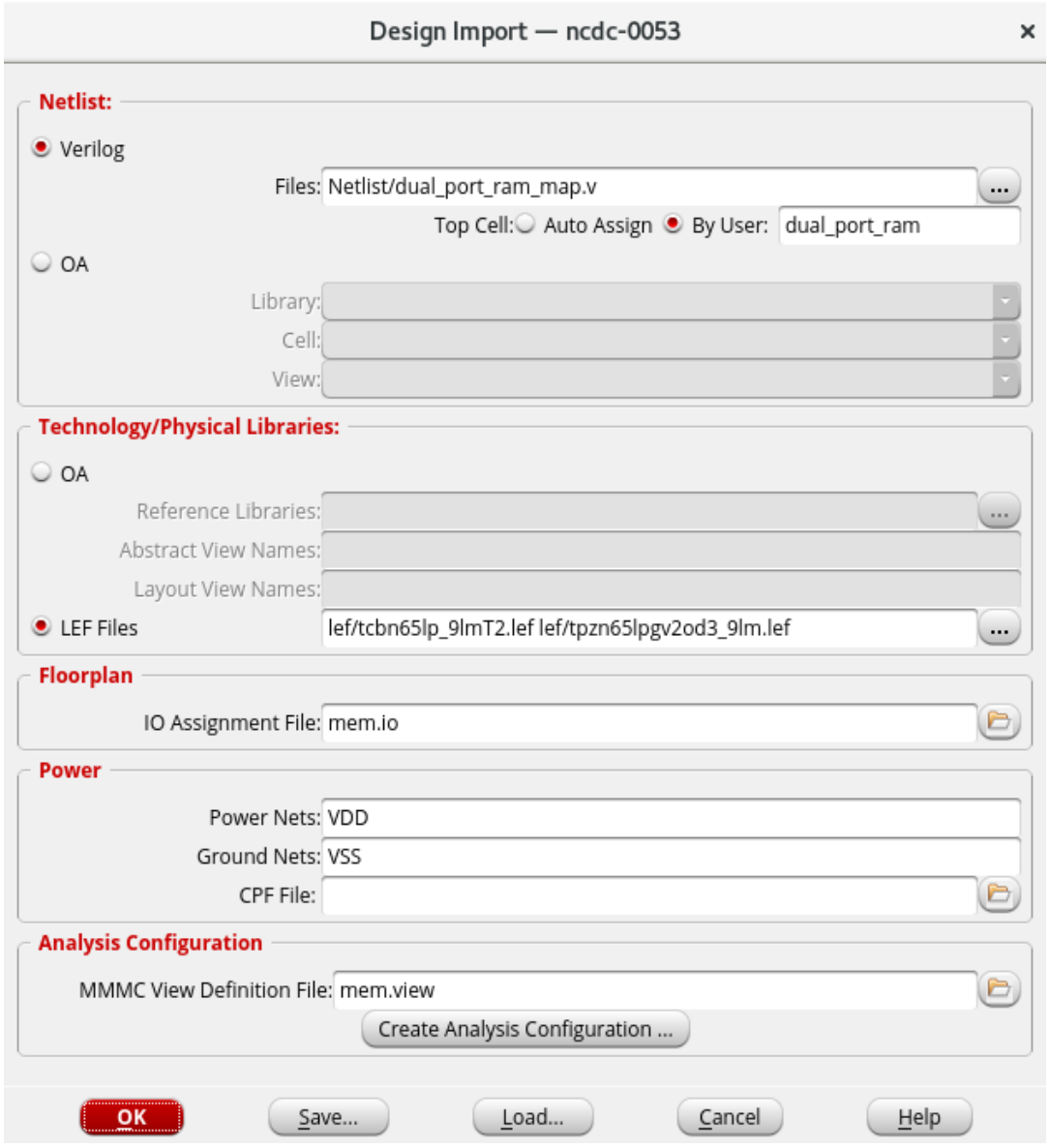
#-------------------------------------------------------------------------------------------
# Timing Point   Flags   Arc   Edge   Cell     Fanout Load Trans Delay Arrival Instance
#                                                (fF)  (ps)  (ps)   (ps)  Location
#-------------------------------------------------------------------------------------------
  comp_en        -       -     F     (arrival)     33 27.6     0     0    1750    (-,-)
  g3333/ZN       -       I->ZN R     INVD2         96 96.0   394   215    1965    (-,-)
  g3209__8428/Z  -       B2->Z R     AO22D0         1  0.0    32   130    2094    (-,-)
  data1_out[17]  -       -     R     (port)         -    -     -     0    2094    (-,-)
#-------------------------------------------------------------------------------------------
```

Back End

Placement & Route

After obtaining the netlist, I did the back-end process of this design. So, in order to do that I ensure following files exist in my directory:

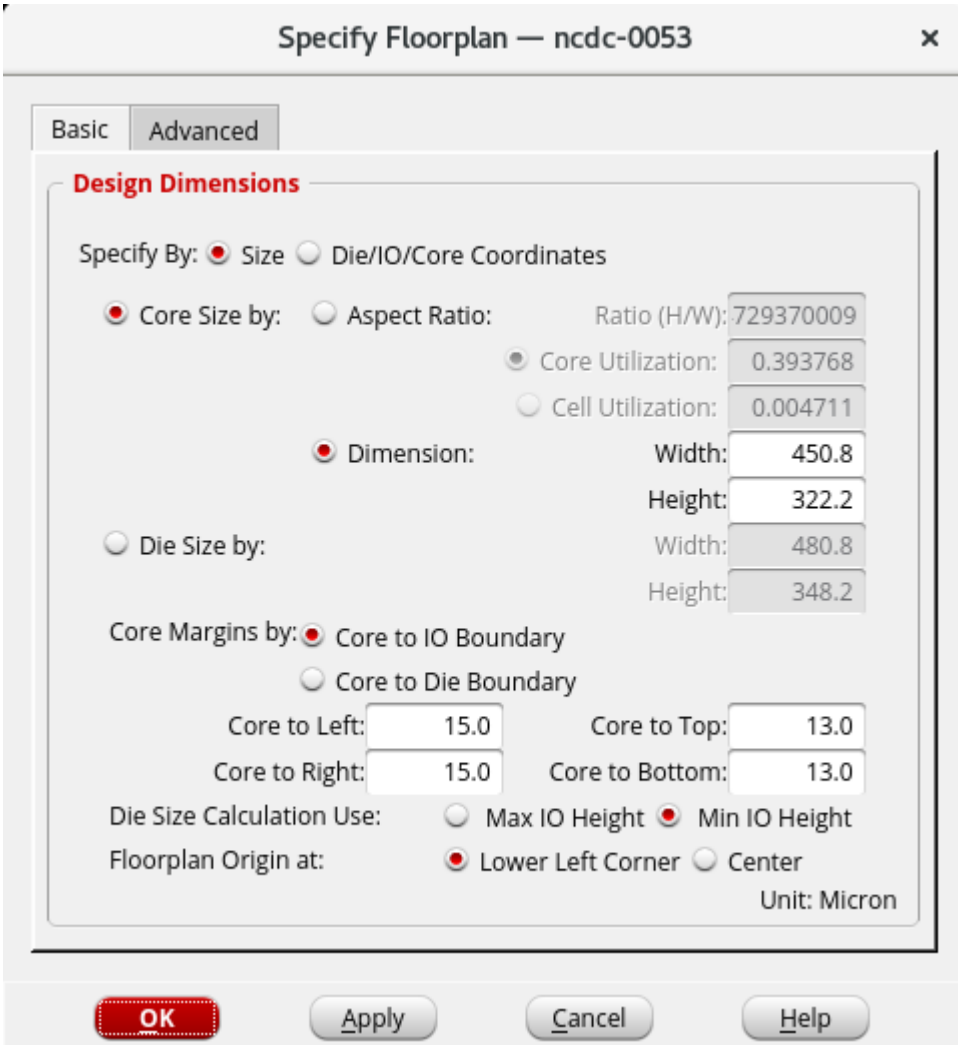
- Top_Wrapper_Netlist.v
- tcbn65lp_9lmT2.lef
- tpzn65lpgv2od3_9lm.lef
- IOfile.io
- MMMCfile.view



FloorPlanning

It is a critical stage in the physical design flow where you define the physical organization and layout of the entire chip before placement. It sets the blueprint for how standard cells, macros, IO pads, and power structures will be arranged on the silicon die. It defines:

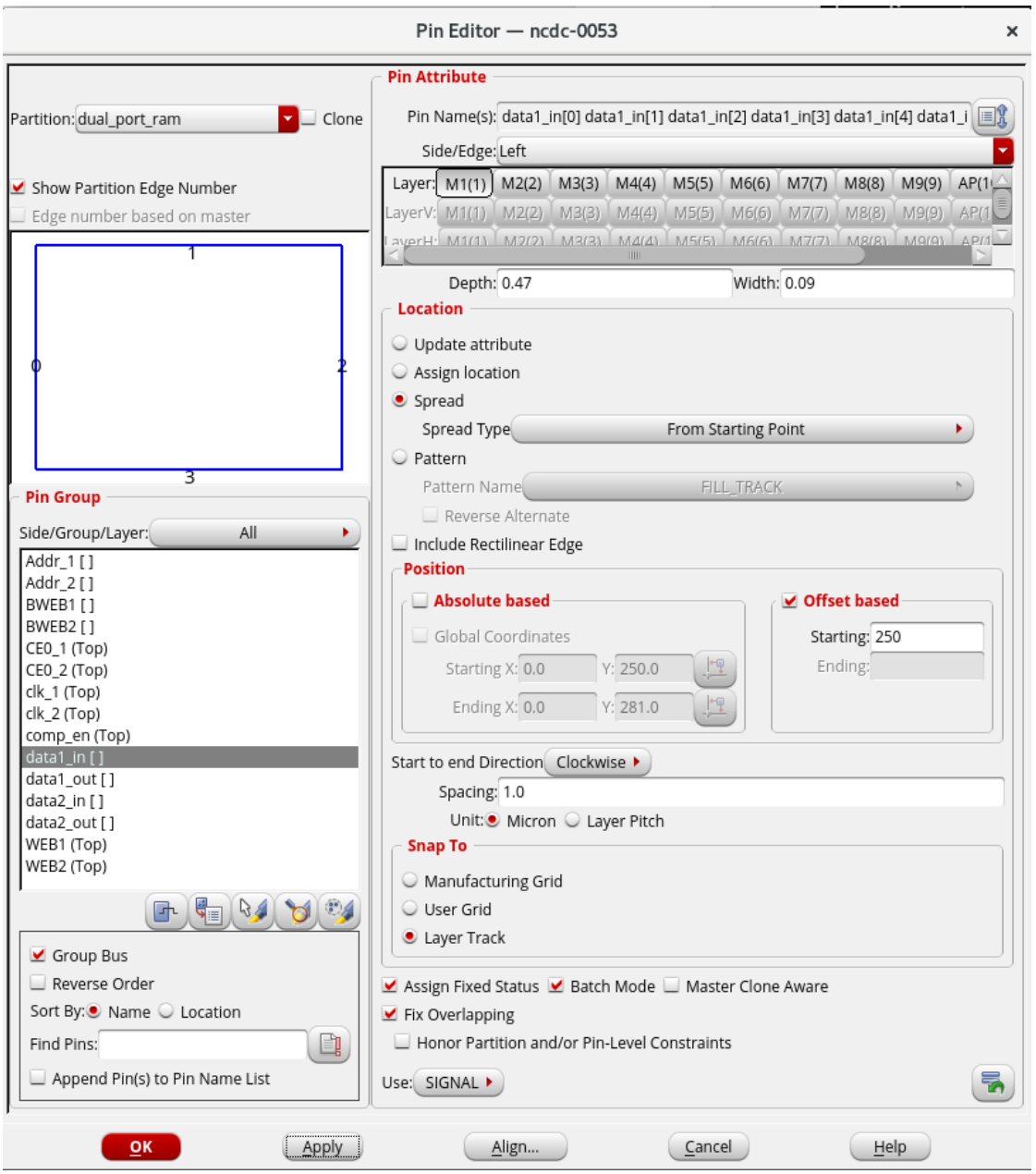
- Core area
- Die area
- Macros/IPs
- IO pins
- Core utilization
- Aspect ratio



After Floor Planning:

Physical





I edited the pins using pin editor.

For Bottom Side:

I use address 1 and address 2 and appropriate offset and keep clockwise direction.

For Top side:

I use BWEB1, BWEB2, CEO_1 top, CEO_2 top, clk_1 top, clk_2 top and comp_en top and WEB1 top and WEB2 top using suitable offset and keeping clockwise direction.

For Left side:

I use data 1_in for input pins and data 1_out for output pins and keep a suitable offset and clockwise direction.

For right Side:

For this, I use data 2_in for input pins and data 2_out for output pins and use appropriate offset and clockwise direction.

After Editing Pins:



This is the initial floor plan of the chip with just IO pins assigned. Looking closely, you can see how different groups of pins are arranged nicely across the boundary of the chip. This can be achieved using the pin editor option under edit task bar.

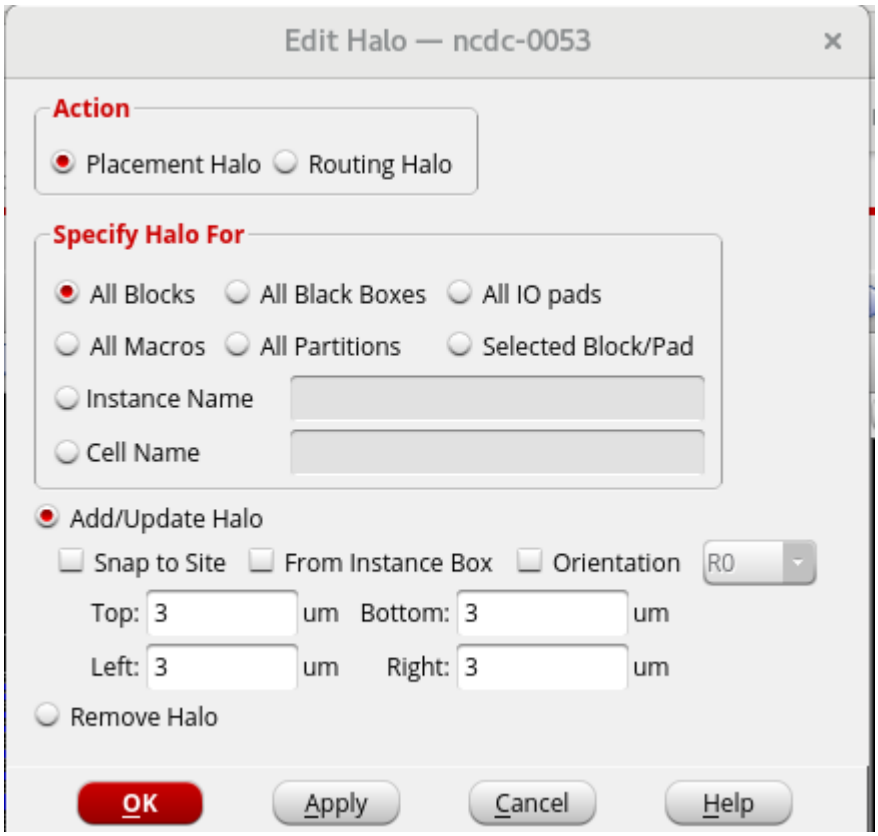
Next step is to place the macro on floorplan area; we will use the following commands for this:
placeInstance dp_ramu1 {150 150} R0 ==== Place the macro at coordinates (x, y) without rotation
set_db inst:dp_ramu1 .place_status fixed ==== Fix the macro so placer won't move it
get_db inst:dp_ramu1 .place_status ==== This reports placement status of macro

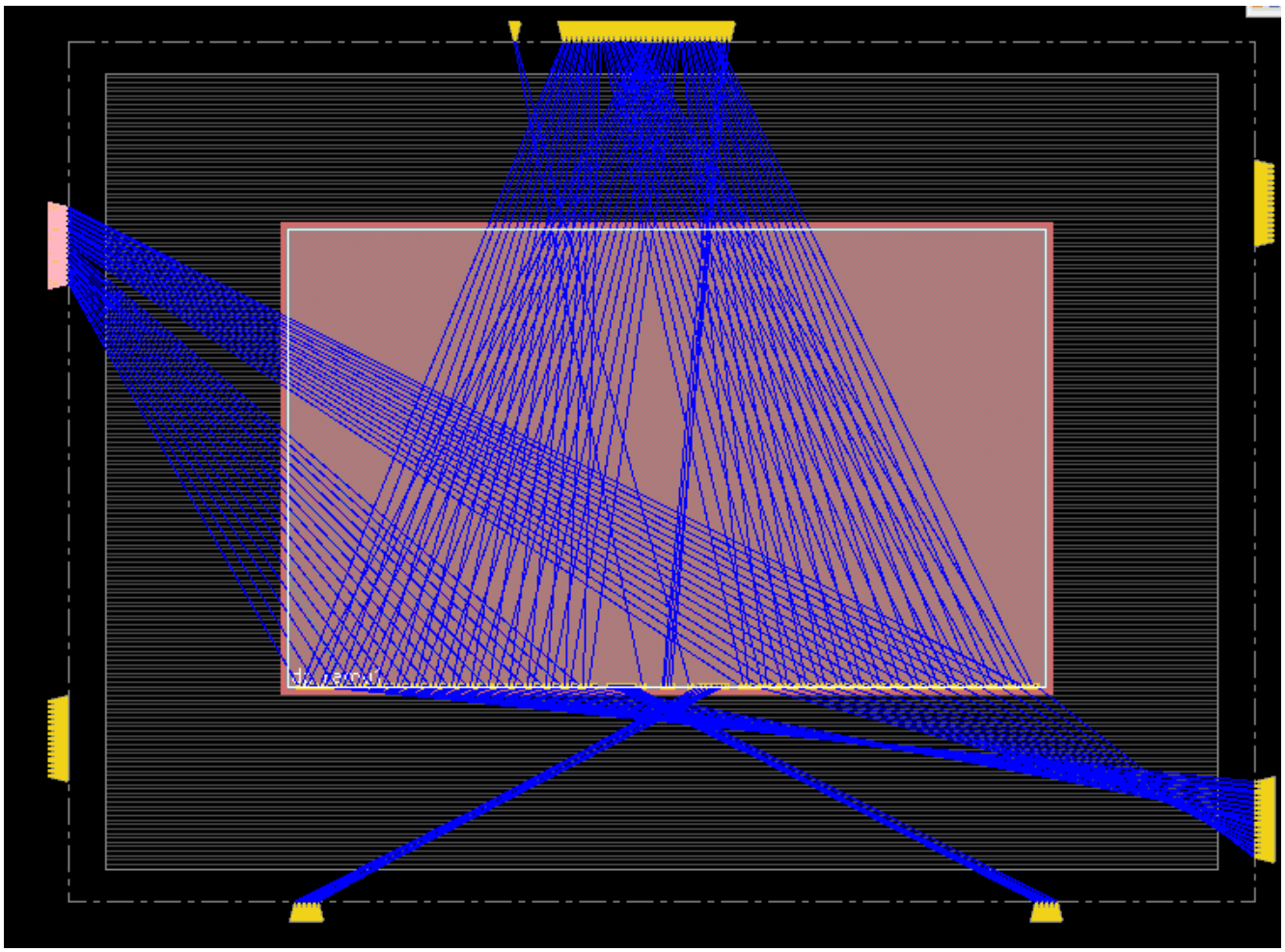
```
innovus 2>
innovus 2> placeInstance dp_ramu1 {150 150} R0
innovus 3> set_db inst:dp_ramu1 .place_status fixed
1 fixed
innovus 4> get_db inst:dp_ramu1 .place_status
fixed
```

In my Design, the macro is placed at the top right corner of the core. So I manually move the macro within the core area at the appropriate placement location.



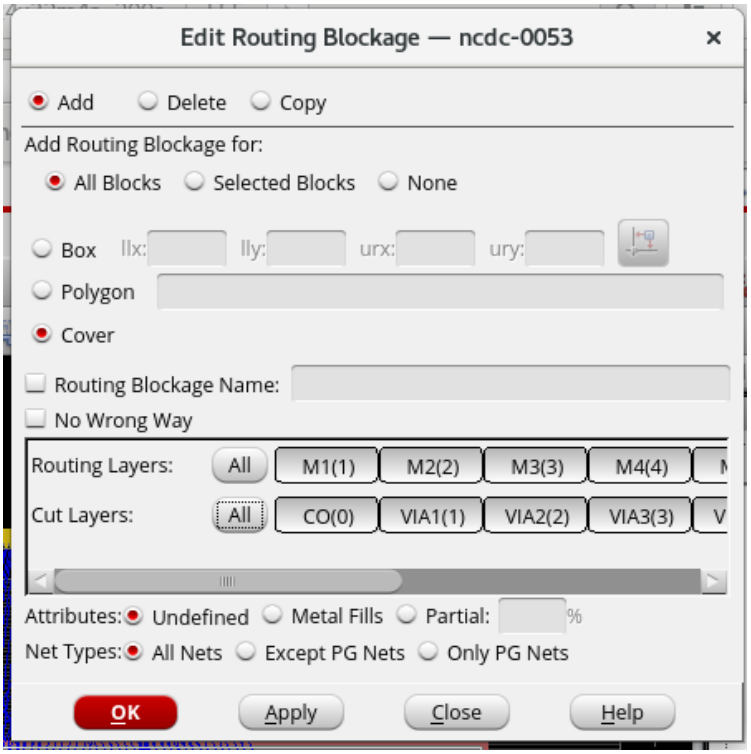
Edit Halo: We can add halo (blockage) around the boundary of the macro so that no std. cells are placed by the tool in close vicinity to its ports.
To add the blockage, simply click on the Floorplan tab → Edit FloorPlan → Edit Halo.



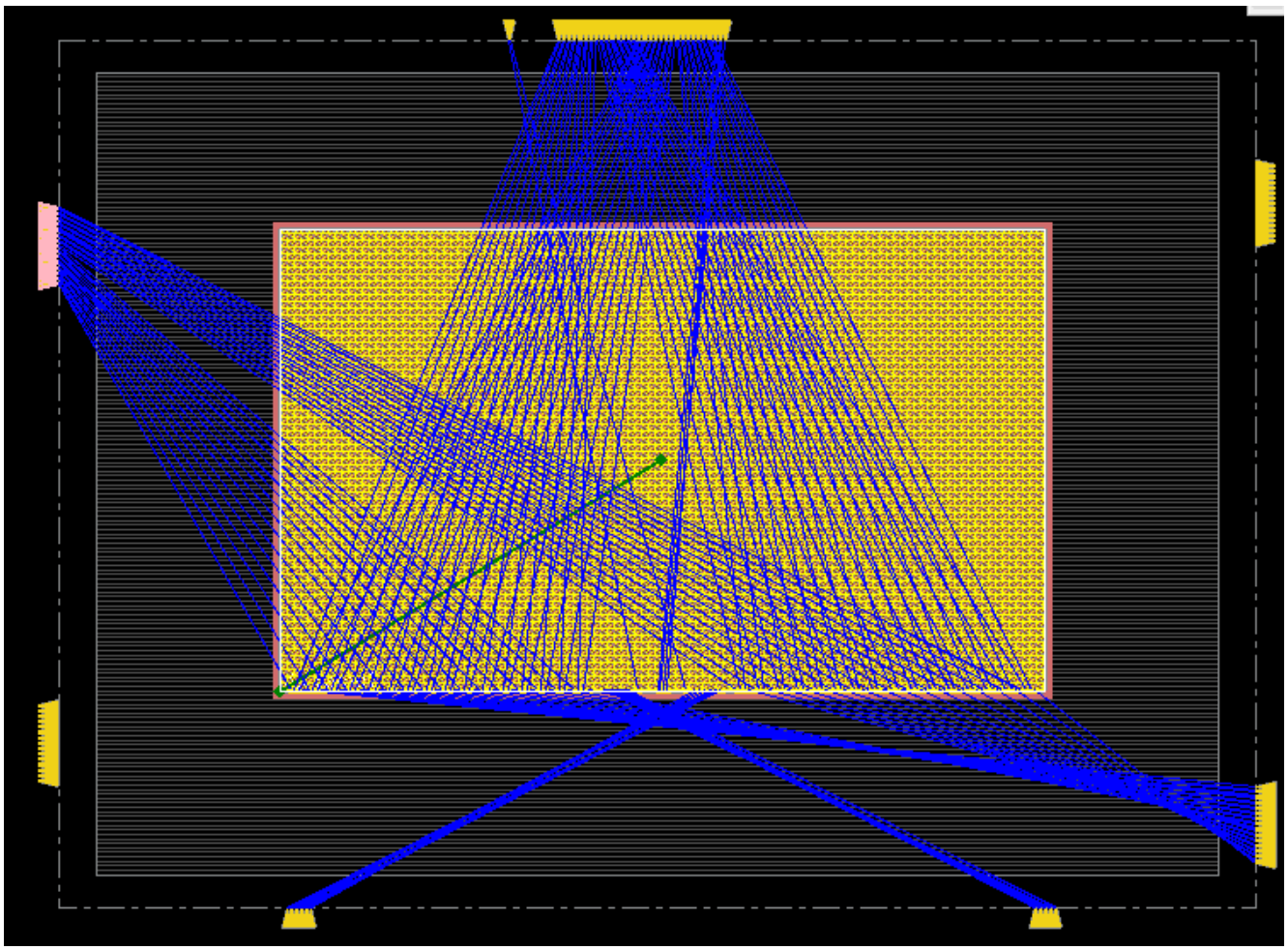


We have added a 3-micron blockage across either side of the macro, which will be std. cell free to avoid any connectivity issues with the internal pins. After this, we will add routing blockage so that we prevent router to create any wiring paths in the region of the floorplan area where macro is placed and is fixed. Aside from the signal routing we also prevent power/via routing in that area.
To add routing blockage, simply do Floorplan tab → Edit FloorPlan → Edit Routing Blockage.
In the pop-up menu, I have selected all the routing layers and cut layers across the macro which will be routing free in every layer as shown.

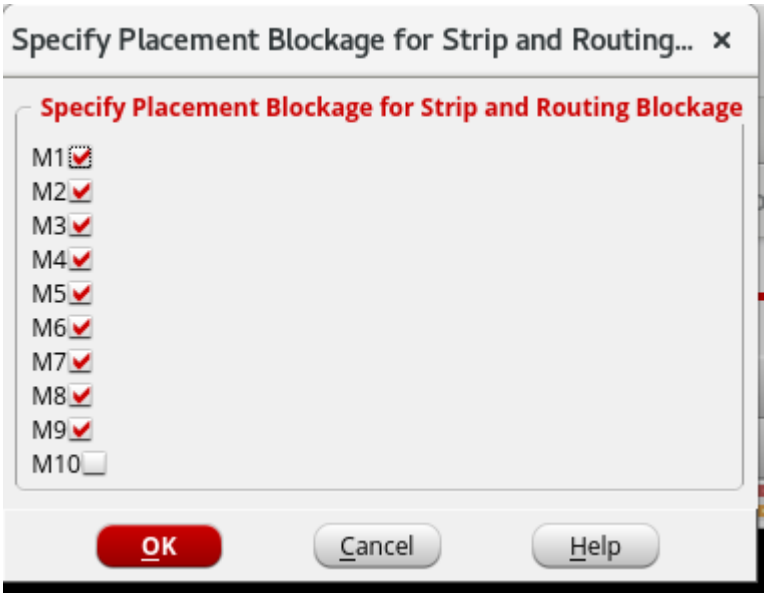
Routing Blockage:



Layout after editing Routing Blockage:



Once we added all the routing and placement blockages in our floorplan, we need to tell the tool to consider these blockages further in the design flow by specifying placement blockages for strip and routing under **Place → Specify → Placement Blockage menu.**



This is done so that the placement tool does not place std. cells under these blockages, so instead of routing blockages they are now treated as placement blockages. Next, we generate the power distribution network (PDN) which powers both the std. cells and the macro. It must be noted that we need to generate the power stripes on the layer above the macro highest metal layer M5 in this case, so power is evenly distributed using power vias down the macro pins. But, before that, we need to create global nets under the power tab.

Adding Global Connects

VDD or VSS Net

- These are the names of the global power and ground nets used throughout the design.
- They connect to the power/ground rails, stripes, and rings.
- Refers to the power/ground pins of standard cells or macros.

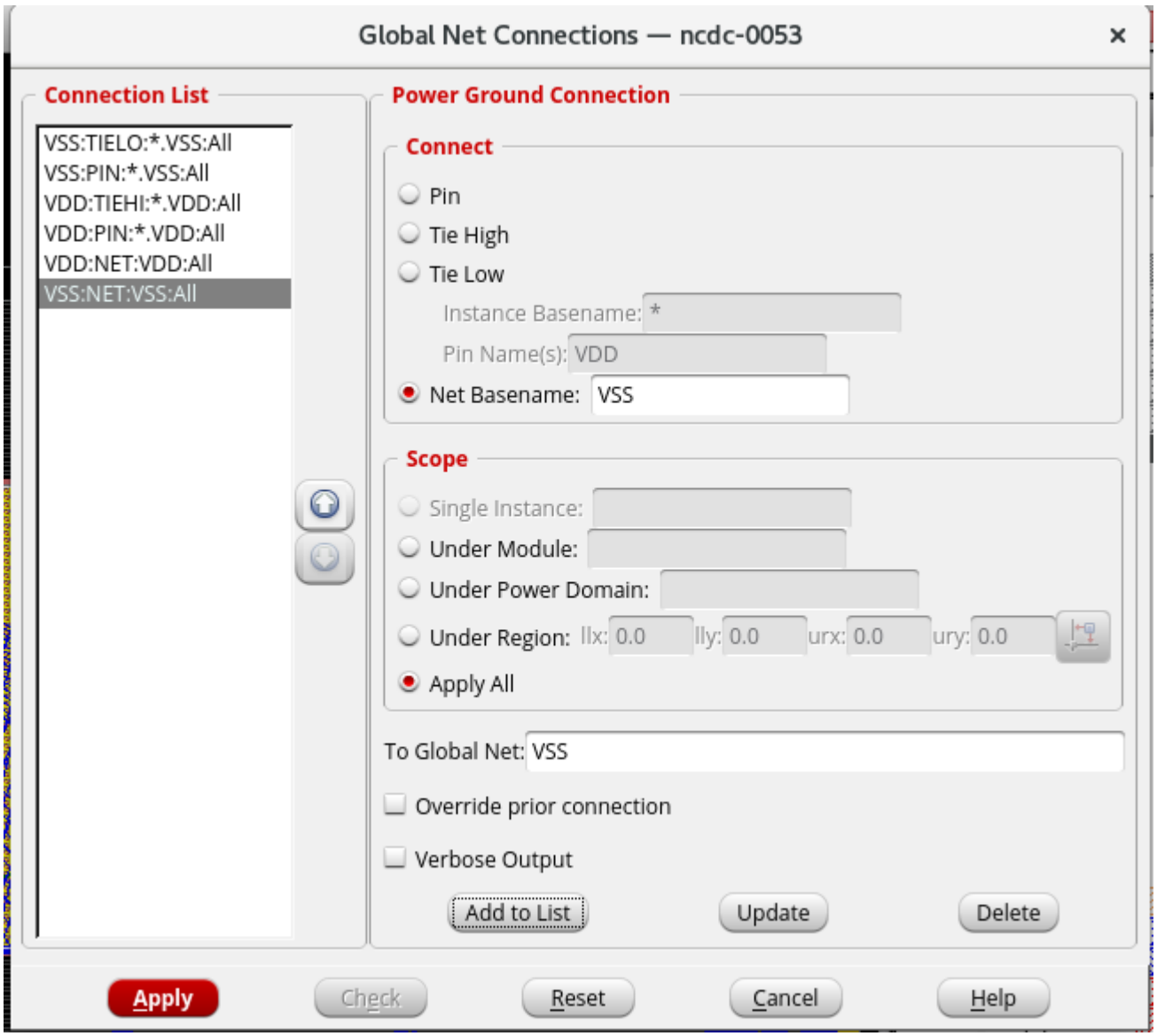
VDD TIEHI & VSS TIELO

- These refer to cells that provide a constant logic high (1) or low (0).
- Used to tie unused inputs or force constant values.

Without it, the tool may:

- Not recognize VDD or VSS connections properly
- Leave cells unpowered
- Fail LVS or IR drop checks
- Cause physical connectivity issues (like stripes/rings not actually powering logic)

For Global connections, click on Power => Connect Global Nets.



Power Planning:

Power planning is an essential part of the ASIC physical design flow, ensuring the chip receives a stable and reliable supply of power to all standard cells, macros, memories, and I/Os.

- Ensure voltage levels remain stable throughout the chip.
- Minimize IR drop (voltage drop due to resistance in metal).
- Avoid electromigration (metal damage due to high current).
- Provide robust power to all functional blocks and corners.

PDN is the actual physical implementation of the power plan. A hierarchical network of wires that deliver power from the chip's IO pads (or bumps) to every internal standard cell and macro. **PDN Includes:**

IO pads → Core rings → Power stripes → Standard cells

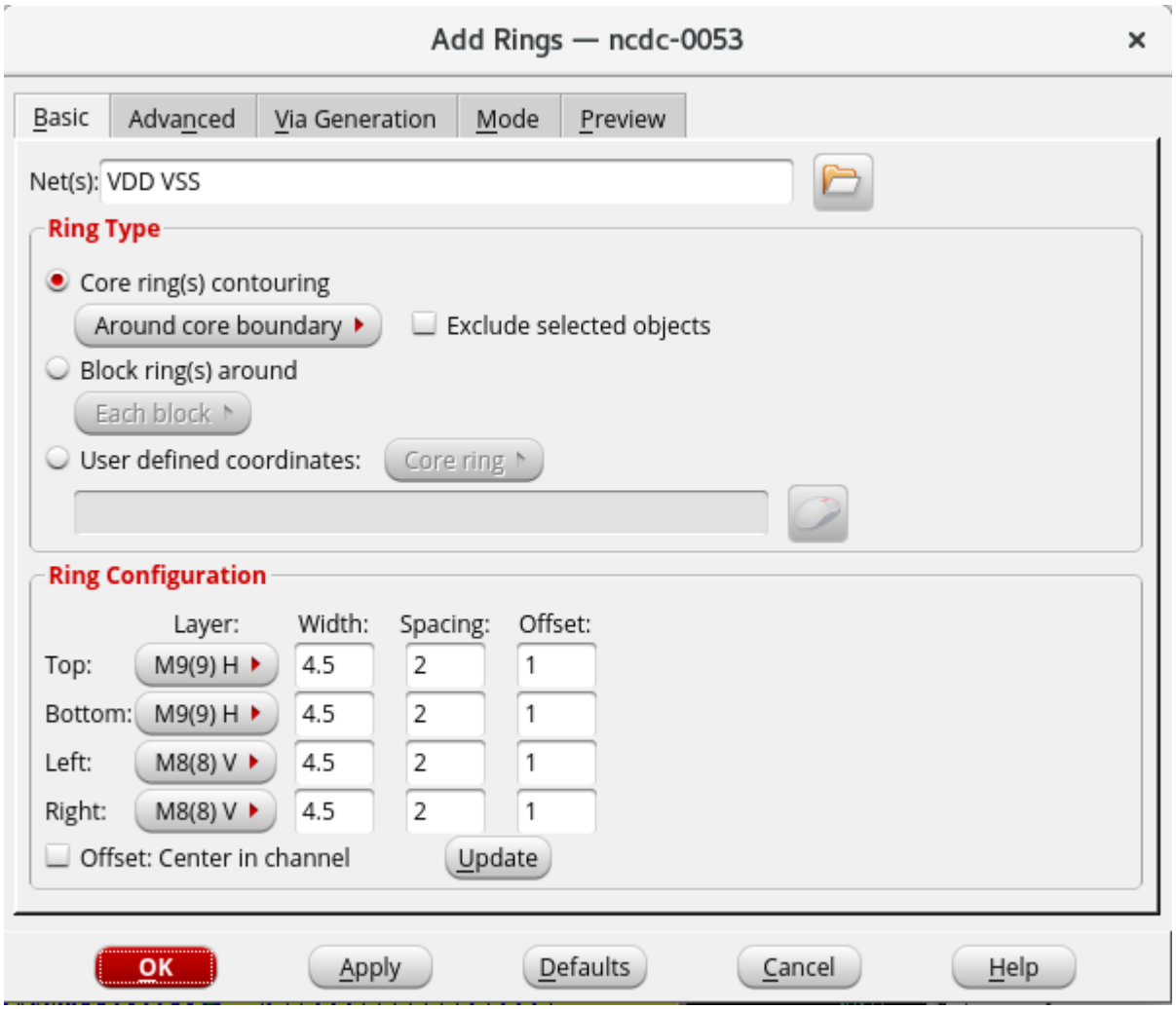
Multiple metal layers used to reduce resistance Connections to decoupling capacitors to stabilize power

Now I have clicked on Power => Power planning => Add Ring.

Added Nets (e.g. VDD, VSS)

For the power ring higher metal layers are chosen such as metal layer 5 and 4 for wider tracks, low

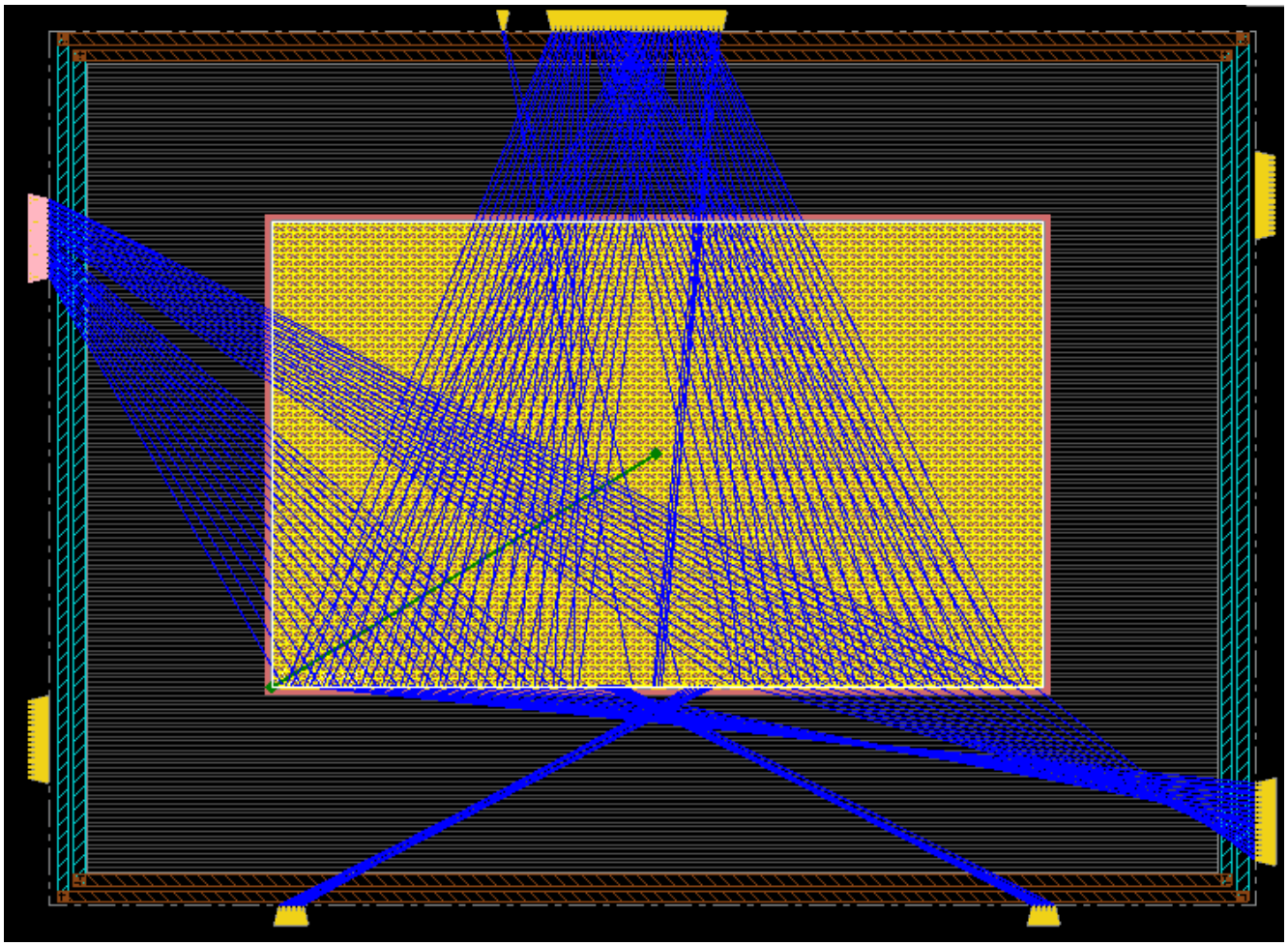
resistance and higher current capacity with low IR drop.
Spacing b/w rings, width and offset from boundary is set in compliance with DRC rule check given by foundry.




```
addRing created 8 wires.
ViaGen created 8 vias, deleted 0 via to avoid violation.
+--------+----------------+----------------+
|  Layer |     Created    |     Deleted    |
+--------+----------------+----------------+
|   M8   |        4       |       NA       |
|  VIA8  |        8       |        0       |
|   M9   |        4       |       NA       |
+--------+----------------+----------------+
innovus 5> The ring targets are set to core/block ring wires.
addRing command will consider rows while creating rings.
addRing command will disallow rings to go over rows.
```

Explanation:
This output shows that the power and ground rings were successfully generated around the core in Innovus. The tool created 8 metal wires (4 on M8 and 4 on M9) and 8 vias (VIA8) to connect these layers for proper power distribution. No vias were deleted, meaning there were no design rule violations. The message also confirms that the addRing command avoided overlapping cell rows and ignored shorts during ring creation. Overall, this indicates that the power ring structure was created correctly and without errors.

After Adding Rings



```
addRing command will ignore shorts while creatir

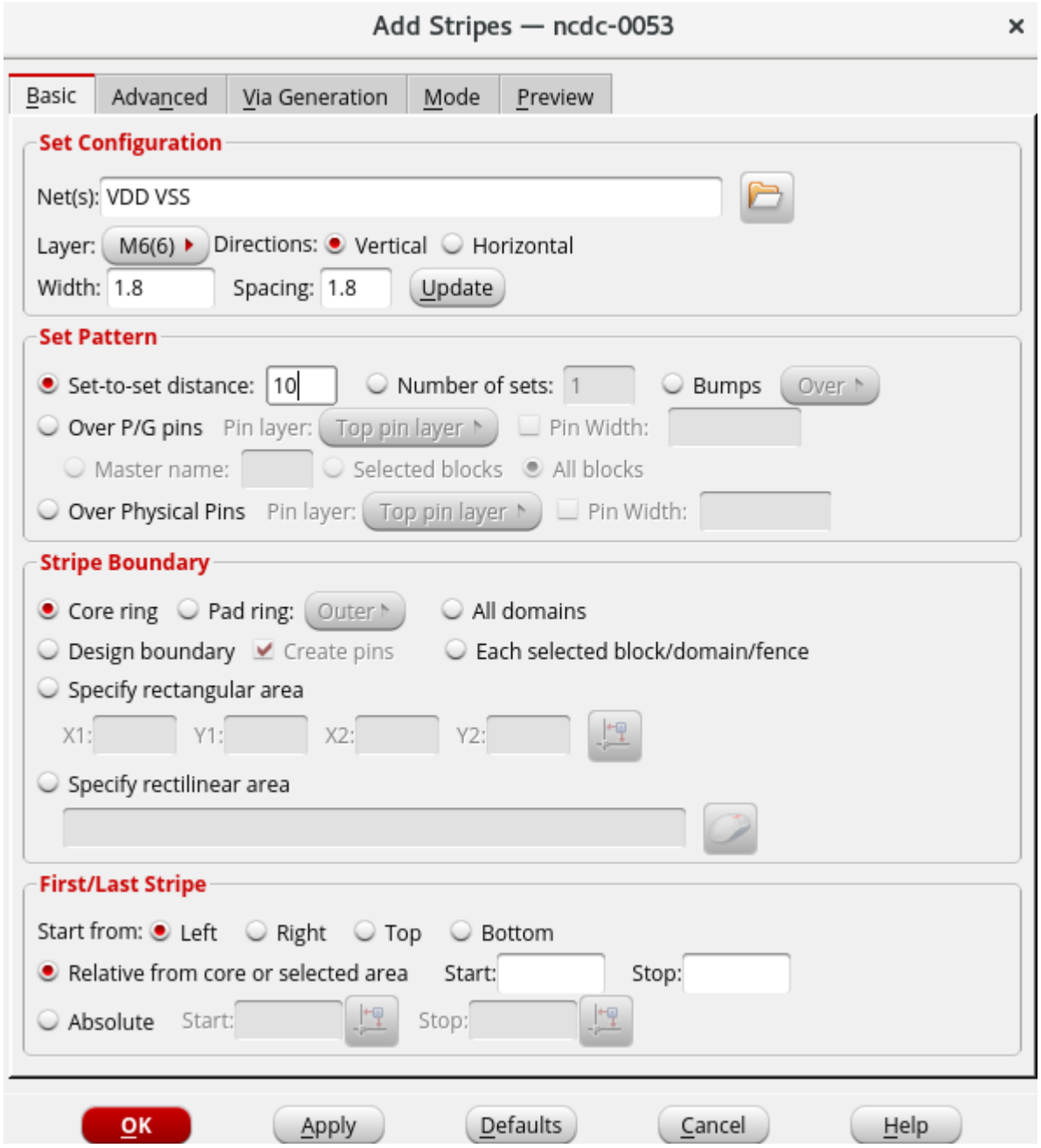
Loading cell geometries (cpu: 0:00:00.0, real: 0
Ring generation is complete.
vias are now being generated.
addRing created 8 wires.
ViaGen created 8 vias, deleted 0 via to avoid vi
+--------+----------------+----------------+
|  Layer |     Created    |     Deleted    |
+--------+----------------+----------------+
|   M8   |        4       |       NA       |
|  VIA8  |        8       |        0       |
|   M9   |        4       |       NA       |
+--------+----------------+----------------+
```

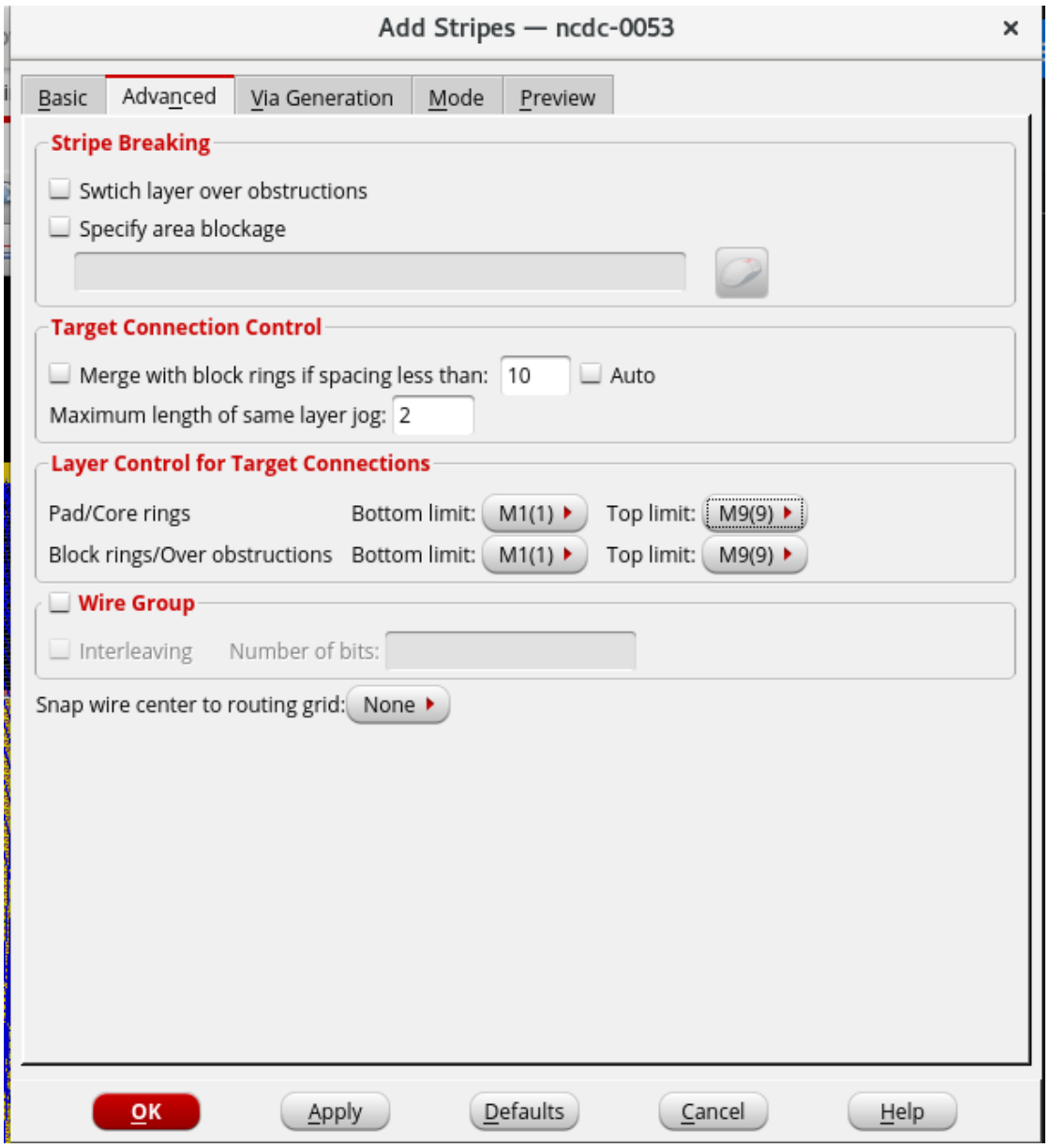
Add strips:

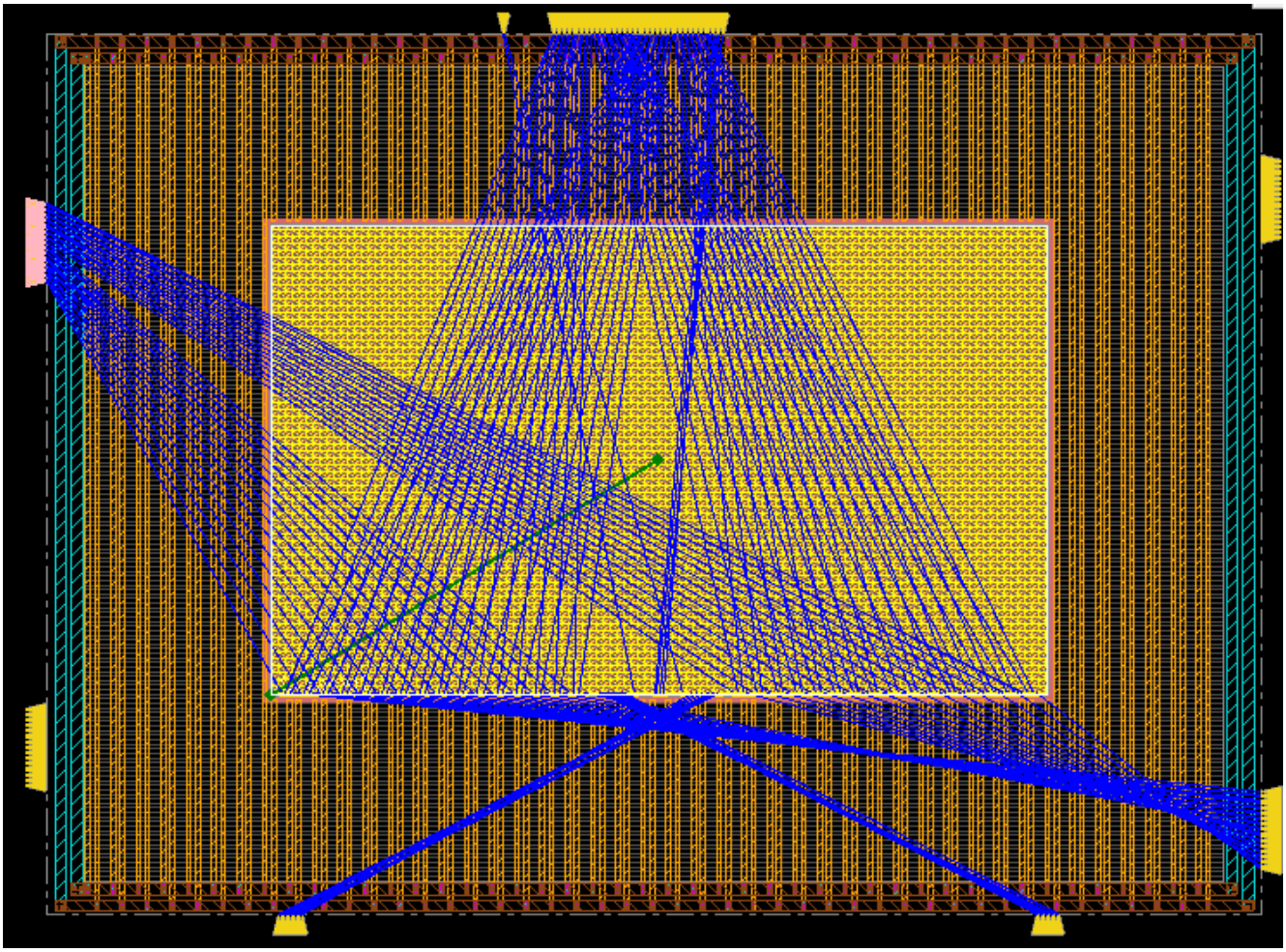
After placing a power ring around the core, metal stripes (VDD and GND) are inserted horizontally and vertically inside the core to:

- Reduce IR drop (voltage drop).
- Provide multiple power taps to standard cells.
- Support current demand evenly across the layout.

For power strips, I have clicked on power => Power planning=> Add strips



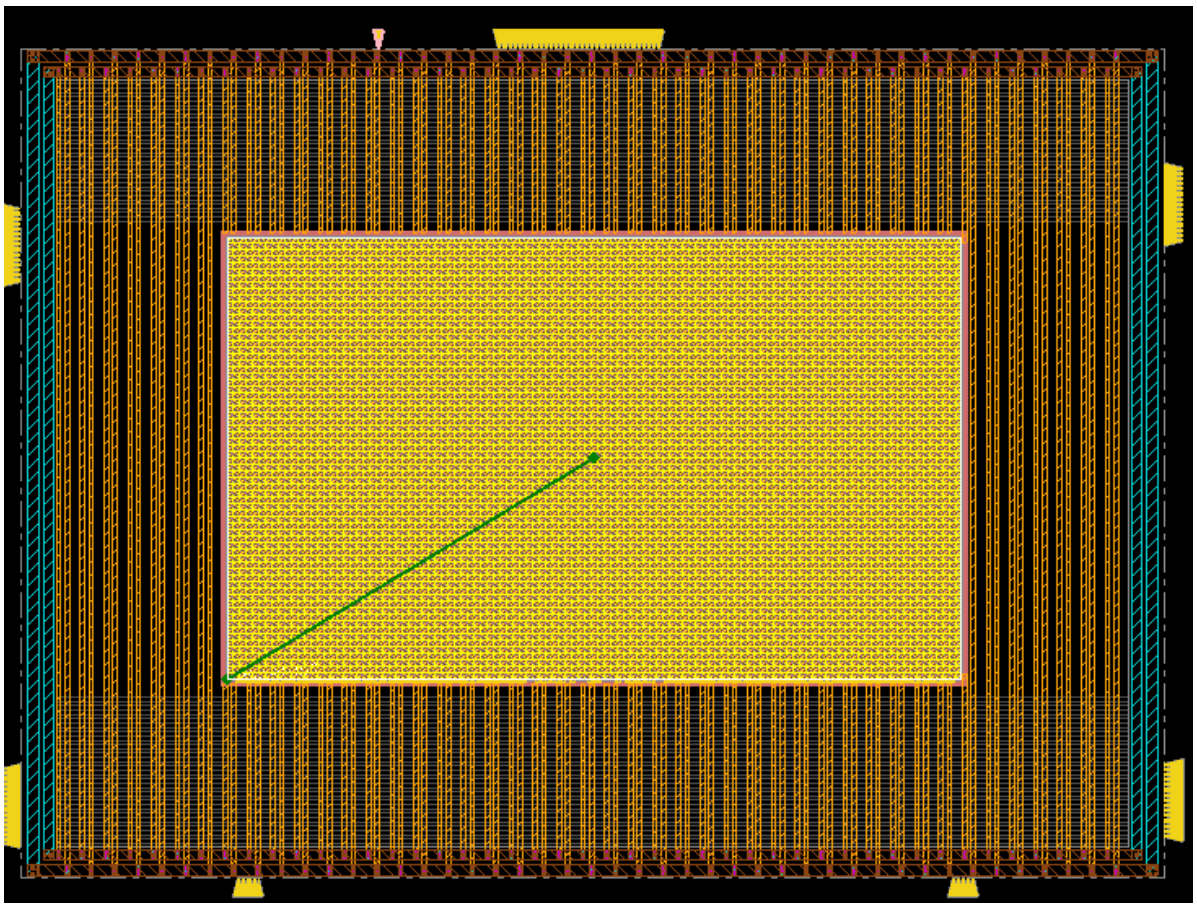

Add Stripes — ncdc-0053
Basic
Advanced
Via Generation
Mode
Preview
Stripe Breaking
Swtich layer over obstructions
Specify area blockage
Target Connection Control
Merge with block rings if spacing less than:
10
Auto
Maximum length of same layer jog:
2
Layer Control for Target Connections
Pad/Core rings
Bottom limit:
M1(1)
Top limit:
M9(9)
Block rings/Over obstructions
Bottom limit:
M1(1)
Top limit:
M9(9)
Wire Group
Interleaving
Number of bits:
Snap wire center to routing grid:
None
OK
Apply
Defaults
Cancel
Help



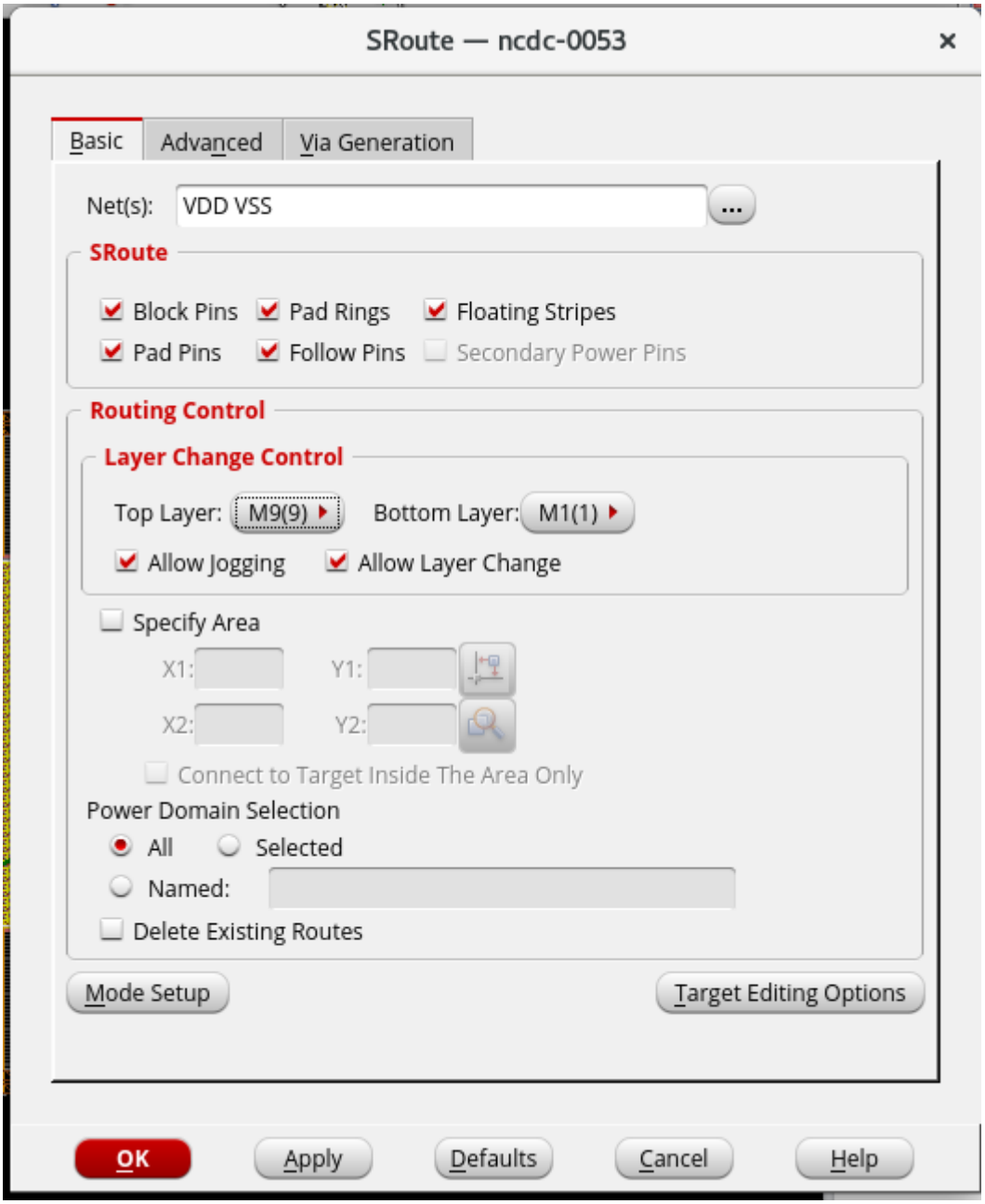
Before, we proceed any further with the PDN, we need to cut the rows horizontally across the macro height on each side. This is to ensure we do not encounter any DRC violations when generating power rails as they are laid across the row edges.

To cut the rows, select Floorplan → Row → Cut Core Rows option.

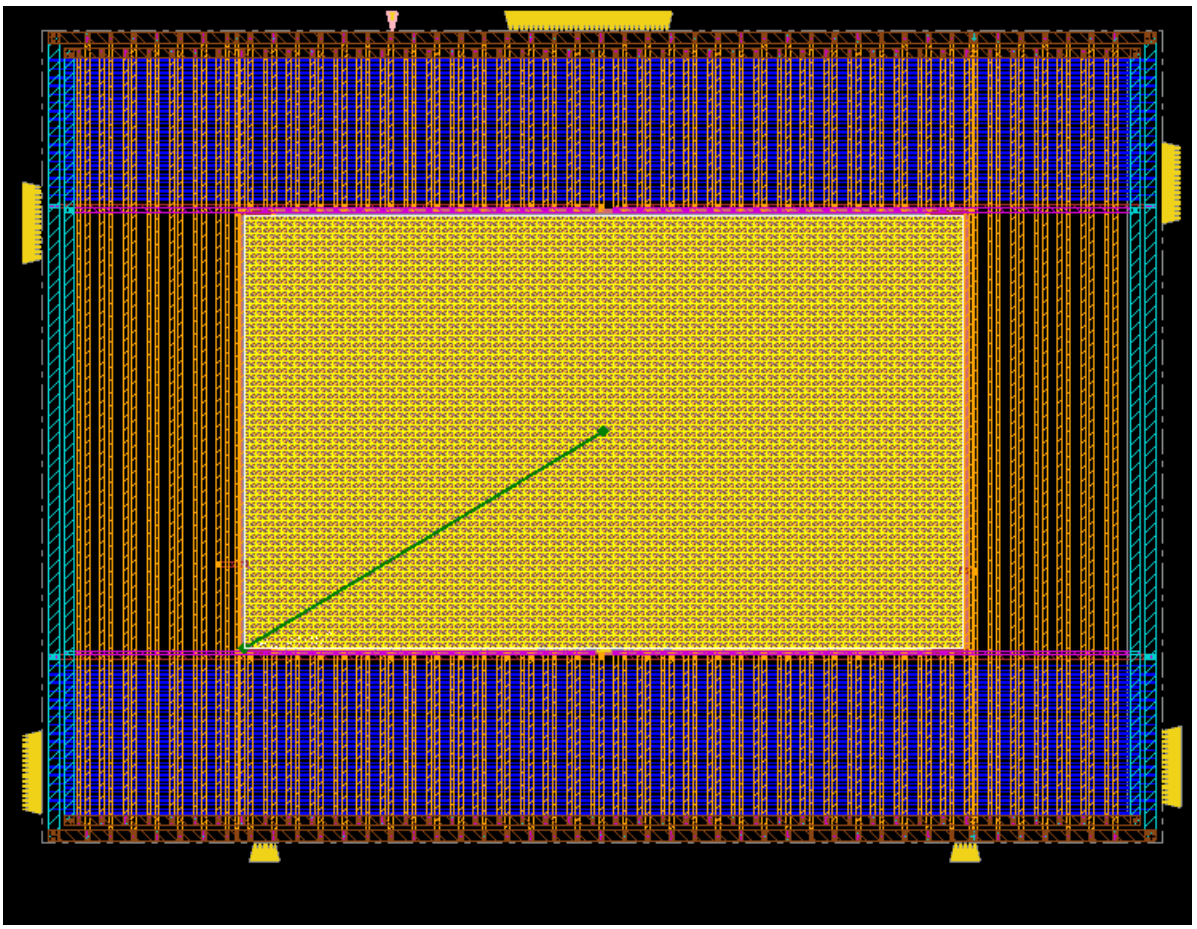
Click on Specified Area, then cursor to draw a box around macro. Again, go to cut core row and click apply



I went to Route -> Special route I used M5 for power ring, so I will select M5 as top layers while specifying power rails in my design. Vias need to be connected from power rails to power rings to get power so select same metal layer you used for power ring.



After Special Routing:



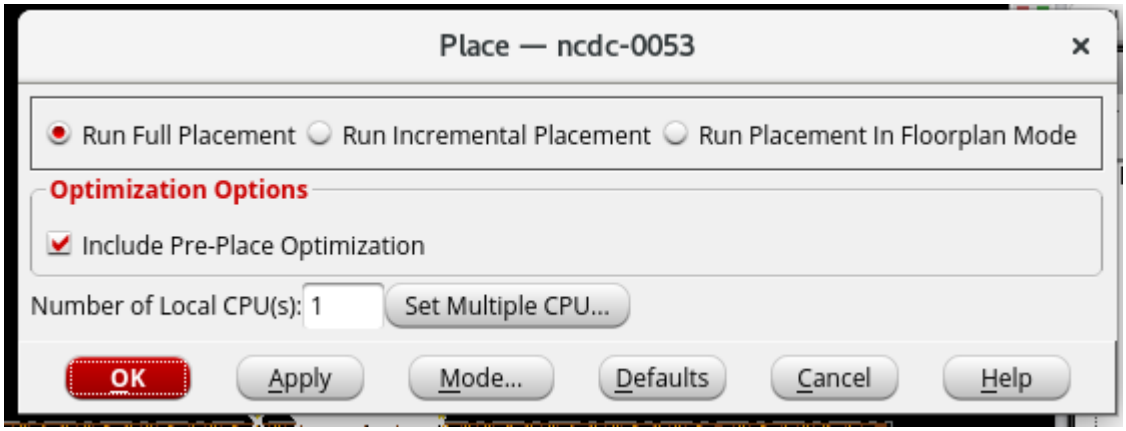
Checking Connectivity:

At this point, check for verify connectivity option and look if there are any dangling VDD or VSS nets. If there are any, manually remove those nets. It should look like the figure below. Ignoring other unconnected nets for the moment.

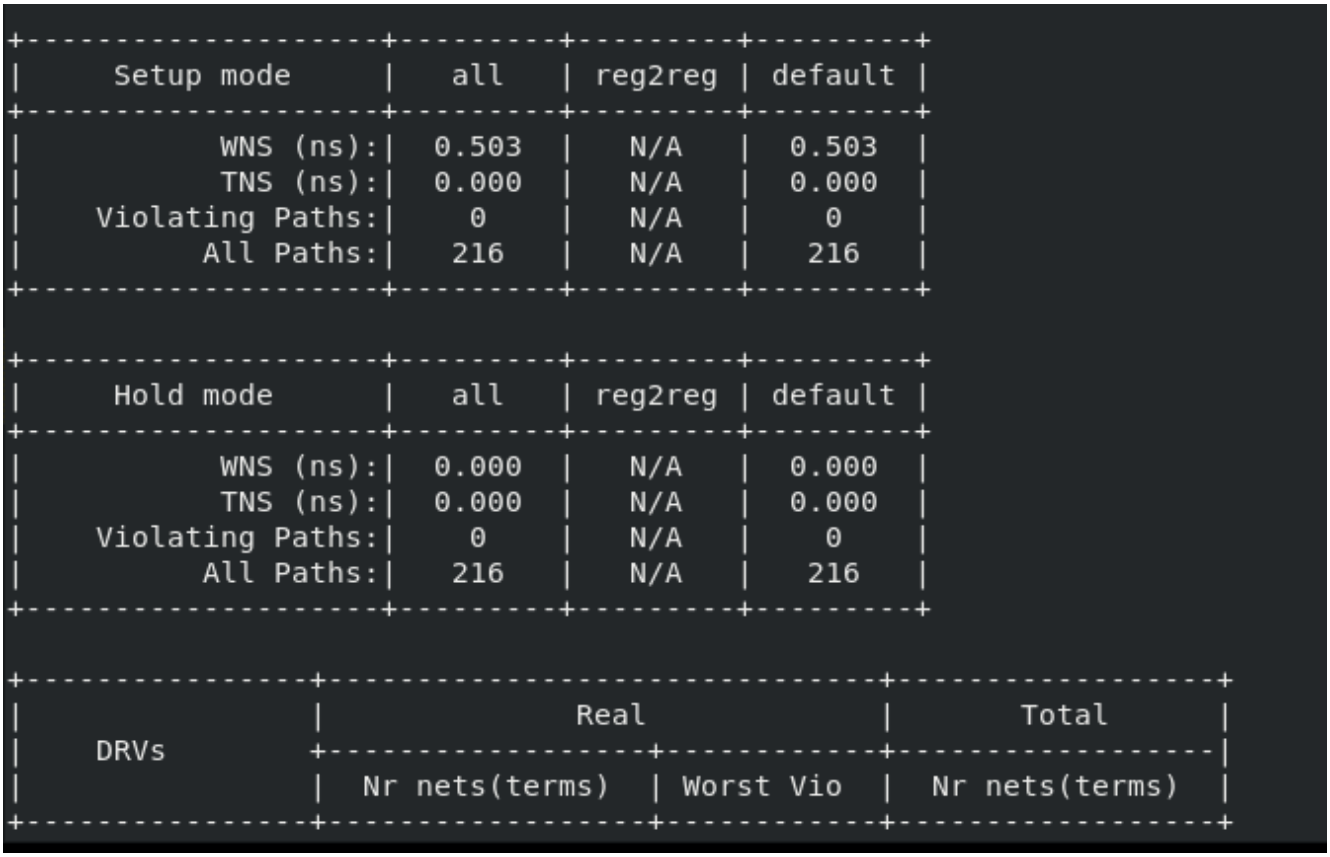
```
Begin Summary
    154 Problem(s) (IMPVFC-98): Net has no global routing and no special routing.
    197 Problem(s) (IMPVFC-94): The net has dangling wire(s).
    351 total info(s) created.
End Summary
```

Placement:

Placement is the stage in the physical design flow where the standard cells (logic gates) are assigned physical locations on the silicon chip, without overlapping, based on the logical netlist generated from synthesis. Placement Now that our floorplan design is ready, we can begin with global and detailed placement. The macro position and orientation will be unchanged during the rest of the design flow. However, std. cells will be added to the rows.



Clock Tree Synthesis:



Setup mode	all	reg2reg	default
WNS (ns):	0.503	N/A	0.503
TNS (ns):	0.000	N/A	0.000
Violating Paths:	0	N/A	0
All Paths:	216	N/A	216

Hold mode	all	reg2reg	default
WNS (ns):	0.000	N/A	0.000
TNS (ns):	0.000	N/A	0.000
Violating Paths:	0	N/A	0
All Paths:	216	N/A	216

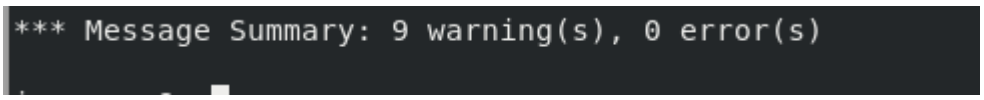
DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)

Routing:
Routing is a key step in digital physical design where the electrical connections (wires) between the placed standard cells and macros are physically created using available metal layers on the chip.

Global Routing Estimates the best routing paths based on congestion and available routing resources.
Detailed Routing Performs the actual physical connection of wires between pins and cells.

- Ensure all nets are connected without open or short circuits.
- Avoid DRC violations (e.g., spacing, width).
- Minimize congestion, delay, skew, and power.
- Maintain timing closure: wires shouldn't cause setup/hold violations.



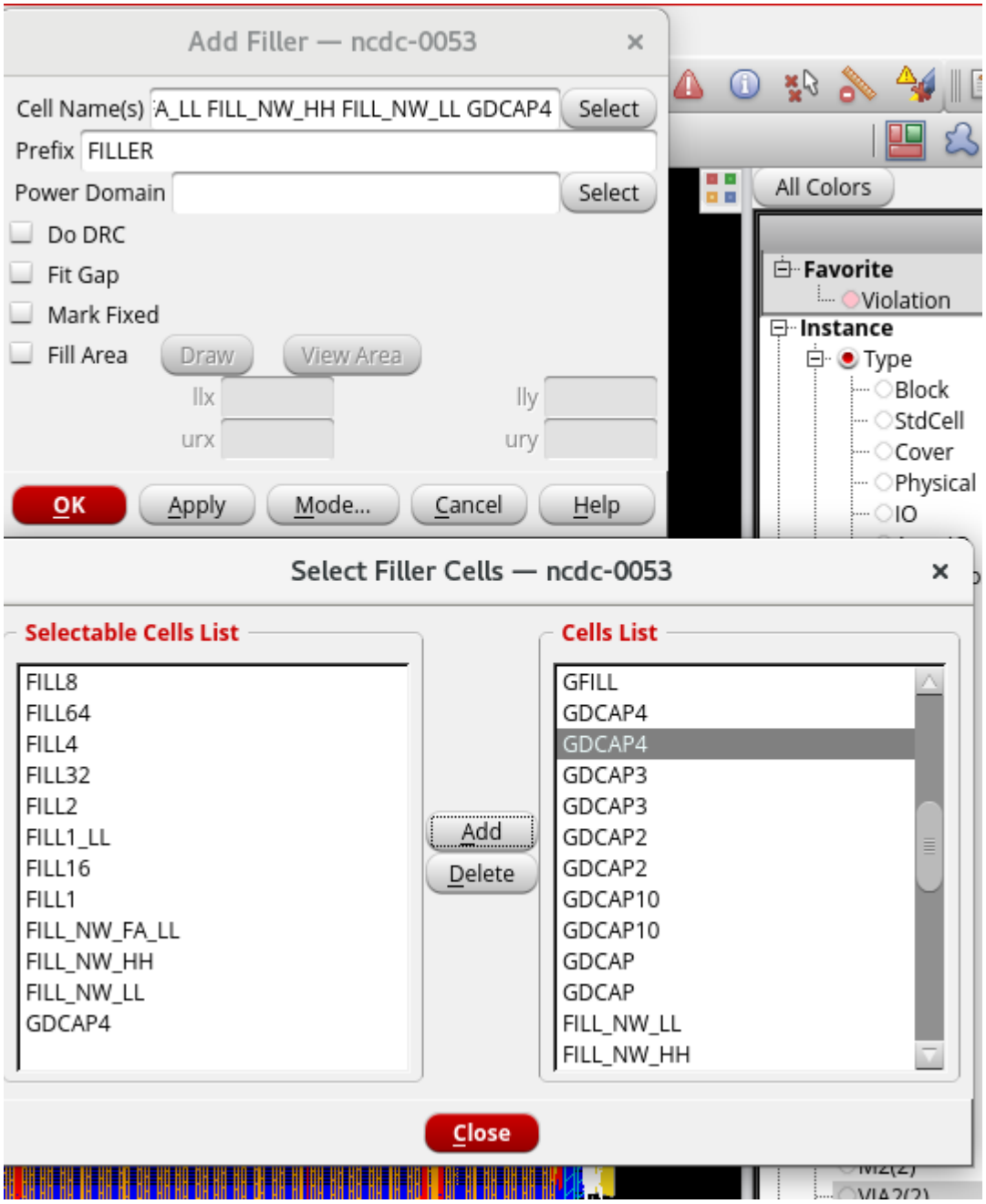


Adding Fillers:

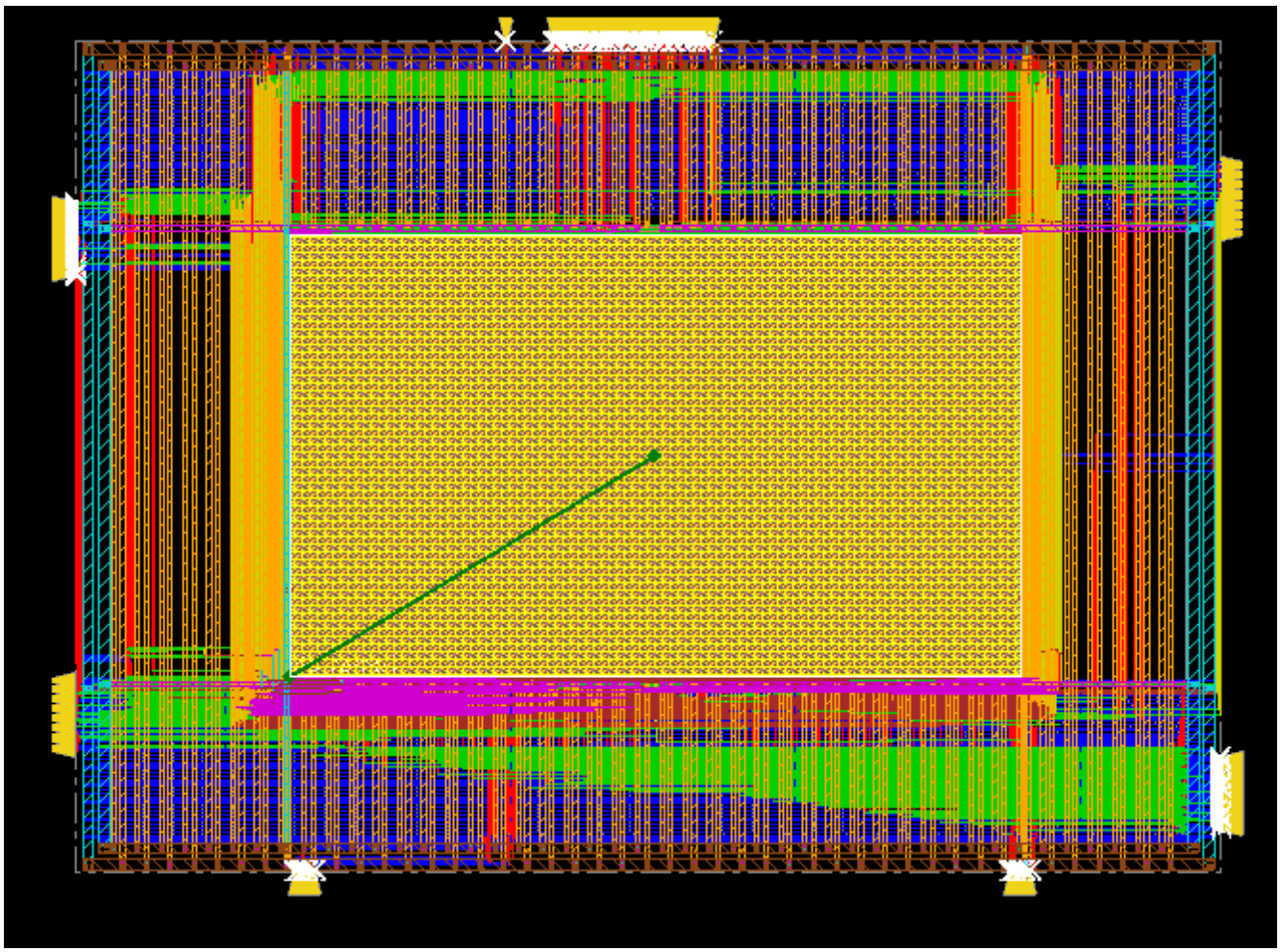
Filler cell placement is the process of inserting special, non-functional standard cells (called filler cells) into the gaps between placed standard cells after place and route (P&R) is complete.

- Filler cells help maintain continuity and substrate tap spacing.
- They ensure electrical connectivity to VDD/VSS for body biasing and prevent latch-up.
- Foundries have rules requiring minimum metal density. Gaps in layout can violate these. filler cells help maintain uniform metal coverage.
- Standard cells share diffusion and poly with neighbors. If gaps are left unfilled, open diffusion regions may result in leakage or manufacturing issues.
- Without filler cells, you might get DRC errors related to well proximity, diffusion, or density violations.

Now I Go to Place -> Physical cell -> Add filler



Physical



Checking DRC:

```
VERIFY DRC ...... Sub-Area : 23 complete 0 Viols.
VERIFY DRC ...... Sub-Area: {403.200 263.520 480.800 348.200} 24 of 24
VERIFY DRC ...... Sub-Area : 24 complete 0 Viols.

Verification Complete : 0 Viols.

*** End Verify DRC (CPU: 0:00:00.3  ELAPSED TIME: 1.00  MEM: 256.1M) ***
```

cONNECTIVITY:

```
******** Start: VERIFY CONNECTIVITY ********
Start Time: Sun Oct 12 17:36:41 2025

Design Name: dual_port_ram
Database Units: 2000
Design Boundary: (0.0000, 0.0000) (480.8000, 348.2000)
Error Limit = 1000; Warning Limit = 50
Check all nets

Begin Summary
  Found no problems or warnings.
End Summary

End Time: Sun Oct 12 17:36:41 2025
Time Elapsed: 0:00:00.0

******** End: VERIFY CONNECTIVITY ********
  Verification Complete : 0 Viols.  0 Wrngs.
  (CPU Time: 0:00:00.0  MEM: 0.000M)
```

Power Analysis:

Power analysis is the process of estimating and analyzing the power consumption of chip design. To ensure it meets power budgets, doesn't overheat, and is efficient for its target application (especially in mobile and battery-operated systems).

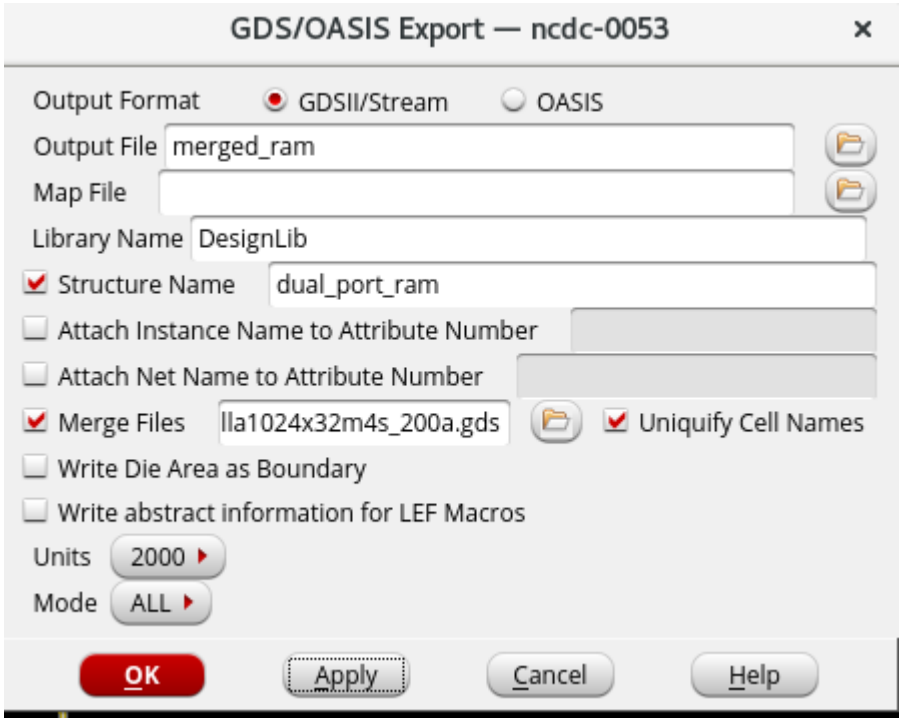
- Ensures power budget compliance
- For thermal management
- To minimize leakage and dynamic power
- To prevent IR drop and EM issues

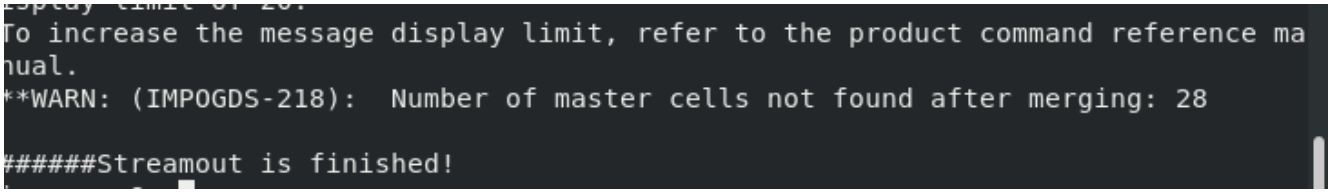
Dynamic power = $\alpha \cdot C \cdot V2 \cdot f$

Static Power: Power consumed when the circuit is idle, due to subthreshold leakage, gate leakage, etc.

Short circuit power: Due to momentary short paths when both PMOS and NMOS are ON during switching.

```
Total Power
-------------------------------------------------------------------------
---------
Total Internal Power:        2.69602982           89.8119%
Total Switching Power:       0.29664060            9.8819%
Total Leakage Power:         0.00919274            0.3062%
Total Power:                 3.00186316
-------------------------------------------------------------------------
---------
```





Critical Analysis:

The lab provides a very practical and industry-oriented experience. It clearly differentiates between soft IP (which is synthesized with your RTL) and hard IP (pre-built black-box macros like SRAM). The manual does a good job explaining how to use different views of the macro (.v, .lib, .lef, .gds, .sp), which is essential for successful integration. The front-end part focuses on creating an updated SDC file, linking the macro's library and Verilog stub, and synthesizing without optimizing the black-box—this step realistically simulates how commercial design houses handle memory integration. The back-end flow is very comprehensive: it covers floorplanning, macro placement, halo and routing blockages, power planning, clock tree synthesis, placement, routing, filler insertion, and DRC/timing verification, followed by GDS export. Each step ensures the student learns how physical design constraints influence the final chip layout.